CHAPTER 13

Reasoning Models and Test-Time Compute

In this chapter

Introduction: The Same Feature, Two Bills . . . 3
13.1 The Second Scaling Axis . . . 4
13.2 What a Reasoning Trace Physically Is . . . 5
13.3 Reasoning Models: Trained For It, Not Asked For It . . . 6
13.4 The Three Knobs . . . 7
13.5 Coverage and Selection: The Decomposition That Explains Everything . . . 8
13.6 The Verifier Asymmetry . . . 8
13.7 Visual Analogies . . . 10
13.8 The Economics: The Most Expensive Tokens You Can Buy . . . 11
13.9 Architecture: Where This Sits . . . 12
13.10 The Claude Surface: Adaptive and Extended Thinking . . . 13
13.11 Effort: The Knob You Will Actually Use . . . 15
13.12 Task Budgets, Thinking Blocks, and the Context Window . . . 16
13.13 Four Compatibility Facts Worth Memorising . . . 17
13.14 Examples: Four Workloads, Four Answers . . . 17
13.15 Code Walkthrough: A Harness That Measures Instead of Asserting . . . 18
13.16 Mini Project: The Test-Time Compute Ledger . . . 19
13.17 Hands-on Labs . . . 25
13.18 Lab Solutions . . . 25
13.19 Best Practices . . . 26
13.20 Common Mistakes . . . 27
13.21 Performance Considerations . . . 29
13.22 Security Considerations . . . 30
13.23 Real Production Examples . . . 30
13.24 Certification Notes . . . 31
13.25 Twenty Practice Questions . . . 32
13.26 Answers and Explanations . . . 33
13.27 Ten Practical Exercises . . . 35
13.28 Five Challenge Labs . . . 35

13.29 Ten Interview Questions 36
13.30 Summary 36
13.31 Glossary 38
13.32 Further Reading 39

Part 2 of 13 · Chapter 13 of 64. Next: Chapter 14 — The Claude Model Family, opening Part 3.

NOTE

Learning Objectives

By the end of this chapter you will be able to:

1. Explain what **test-time compute** is, why it constitutes a second scaling axis, and why Chapter 12's training-compute laws did not predict it.
2. Describe **chain of thought** mechanically — as generated tokens the model then conditions on — and say precisely why that helps without claiming the model is introspecting.
3. Distinguish a **reasoning model**, trained to use the intermediate space, from **prompted chain of thought**, which merely asks for it.
4. Name the three **test-time compute knobs** — think longer, sample more, verify and select — and state what each one buys and what it cannot buy.
5. State the **verifier asymmetry**, decompose an accuracy figure into **coverage** and **selection**, and predict which of your workloads reasoning will help.
6. Price a reasoning trace using Chapter 11's rate card, and explain why thinking tokens are the most expensive tokens a request can generate.
7. Configure thinking on Claude: **adaptive** versus **extended**, the `effort` parameter, task budgets, thinking-block preservation and signatures, and interleaved thinking with tools.
8. Decide, with a measurement rather than an opinion, whether a given workload should buy inference compute, buy a better model, or buy neither.

NOTE

Prerequisites

Chapter 4 for the decode loop and the KV cache, because every reasoning token is one more decode step and one more cache entry. **Chapter 9**, section 9.7, for what a token costs in memory. **Chapter 10** is the chapter this one argues with: its capability triage, its no-introspection limit in section 10.6, its structural limits in 10.10, and the addition network whose weights we reuse. **Chapter 11** for the two-term latency model and the rate card — thinking is billed as output, and output is where both the money and the milliseconds are. **Chapter 12** for scaling laws, for the compute identity `C ≈ 6ND`, and for the discipline that a number without an interval is not a finding.

Chapter 12 closed by saying that the most significant recent change in what more compute buys is the one place the scaling axis moved from training to inference. This is that chapter. It is also the last chapter of Part 2: after this, models stop being objects of study and become components you configure.

Introduction: The Same Feature, Two Bills

A platform team turns on thinking across two pipelines in the same week.

The first is a support-ticket summariser. Roughly a million requests a month, each one a few thousand tokens of conversation in and a short paragraph out. They set `effort` to `high` — the default, and the setting the documentation recommends for quality — and wait for the quality numbers. A week later the numbers arrive. Summary quality, measured by the same rubric and the same human raters as before, is unchanged: the confidence intervals overlap almost exactly. The bill is up by a factor of nine.

The second is a build-fixing agent. It takes a failing test suite, proposes a patch, and applies it. Same model, same `effort`, same week. The pass rate goes from 38% to 71%, and the bill roughly triples.

Same feature. Same setting. One pipeline paid nine times as much for nothing; the other paid three times as much for a near-doubling. Before the end of this chapter you should be able to say which way it will go for a workload of your own, before you turn anything on, and the reason will not be that summarisation is "easy" and patching is "hard". It is that one of those pipelines has something the other does not: **a cheap, automatic way to tell whether an answer is right.** The build agent has a test suite. The summariser has a rubric and a human.

That difference is the spine of this chapter. Everything else — the mechanism, the parameters, the pricing — hangs off it.

We will build the argument in three parts. First, the **mechanism**: what a reasoning trace actually is, why generating tokens before answering changes what a fixed-depth network can compute, and why this is not the model "explaining itself". Second, the **measurement**: a harness that takes one task and spends inference compute on it four different ways, plotting accuracy against tokens, so that the diminishing-returns curve and the verifier asymmetry are measured rather than asserted. It produces a result we did not want and are going to publish anyway: on our task, majority voting over sixty-four samples is *worse* than one greedy sample, at sixty-four times the price. Third, the **surface**: how Claude exposes all of this, verified against the platform documentation, and what each knob costs.

13.1 The Second Scaling Axis

Chapter 12 established a shape. Loss falls as a power law in training compute; the compute identity `C ≈ 6ND` turns a budget into a choice between parameters and data; Chinchilla settled that choice at roughly twenty tokens per parameter. Every term in that framework is fixed before the model is served. By the time you send a request, `N` and `D` are history, the parameters are frozen — the running thread from Chapter 1 — and the only thing you control is the prompt.

Test-time compute breaks that framing.

> DEFINITION
>
> **Test-time compute**
>
> **Test-time compute** (also *inference-time compute*) is the computation spent producing an answer to a single request, as distinct from the computation spent training the model. It is measured in generated tokens, sampled attempts, or wall-clock seconds, and unlike training compute it is spent **per request, by you, at your discretion**.

The observation that reorganised the field is this: for a fixed model, accuracy on hard tasks improves as a function of inference compute, and the improvement follows curves of its own. You can hold `N` and `D` constant, change nothing about the weights, spend more tokens per request, and get a measurably better answer. That is a second axis, and it is the one you own.

Two consequences follow immediately, and they pull in opposite directions.

It is a lever you can pull today. Training scaling requires a training run. Inference scaling requires a parameter change. The feedback loop is a request rather than a quarter.

It is a lever you pay for on every single request, forever. Chapter 12's over-training argument turned on exactly this asymmetry: training is paid once and inference is paid always, which is why Llama 3.1's 8B model was trained to roughly ninety times the compute-optimal token ratio. Test-time compute inverts the trade. It buys accuracy with the expensive currency instead of the cheap one.

Chapter 12's laws did not predict this and could not have. They are laws about **training loss** as a function of **training resources**, fitted on models decoded in the ordinary way: one forward pass per token, answer immediately. Nothing in the framework has a term for "and then let it write four thousand tokens of scratch work first". The axis was not mispredicted; it was absent. This is worth holding onto as an example rather than a curiosity, because it is the standing risk with any empirical law: a scaling law tells you what happens along the axes you varied.

13.2 What a Reasoning Trace Physically Is

Strip away the vocabulary and a reasoning trace is unremarkable.

The model generates tokens. Those tokens are appended to the context. Subsequent tokens are generated conditioned on them, through the same attention mechanism, the same KV cache, the same decode loop that Chapter 4 described. There is no separate reasoning module, no second network, no privileged scratchpad with different rules. Chapter 9's inventory of the seven components of a served model gained no eighth entry.

> DEFINITION
>
> ### Chain of thought
>
> **Chain of thought** is the practice of having a model generate intermediate tokens — working, restatement, candidate approaches, checks — before it generates its final answer, so that the final answer is produced conditioned on that intermediate text.

So why does it work? Three reasons, and they are all mechanical.

It adds serial computation depth. A transformer applies a fixed number of layers per token. Chapter 4's stack is what it is: whatever a forward pass can compute, it computes in that many steps, and no prompt makes it deeper. But the number of *tokens* is not fixed. Generating a token, appending it, and attending to it on the next step is a serial computation — the second step genuinely depends on the first — so a model that emits five hundred intermediate tokens has performed roughly five hundred sequential rounds of layer-depth computation, chained. A model that answers immediately has performed one. For any problem whose solution requires more sequential steps than the network has layers, the intermediate tokens are not a stylistic preference. They are the only place the computation can happen.

It externalises state. The running thread from Chapter 1 is that inference is forward-only and stateless; the only memory is the context window. Intermediate tokens are how a model writes something down. A partial sum, a discarded approach, a constraint it has just derived — none of these survive from one forward pass to the next unless they are written into the context as tokens. Chain of thought is the model using the context window as working memory, which is precisely what Chapter 4 said the context window is.

It moves the distribution. Even at equal depth, the tokens that follow "Let me work through this step by step: first," are drawn from a different conditional distribution than the tokens that follow the bare question. Training data in which careful working precedes correct answers is abundant. Conditioning on the appearance of careful working shifts probability mass towards the answers that usually follow it.

> WARNING
>
> **What the trace is not**
>
> It is **not a report of the computation that produced the answer.** Chapter 10, section 10.6, established that models have no introspective access to their own weights and activations; nothing in this chapter repeals that. A reasoning trace is generated text that *causally influences* the answer, because the answer is conditioned on it. That is a much stronger claim than "decoration" and a much weaker claim than "explanation". The trace can be wrong while the answer is right, right while the answer is wrong, and — this is the uncomfortable one — it can omit the consideration that actually determined the output. Chapter 5 made the same point about attention weights: attention is not explanation. A chain of thought is not a mechanistic account of the forward pass, and treating it as an audit trail is a governance error we return to in section 13.22.

13.3 Reasoning Models: Trained For It, Not Asked For It

Prompted chain of thought — "think step by step" — has been standard practice for years. A reasoning model is a different thing, and the distinction is worth getting exactly right because it is the sort of thing an exam asks about.

Prompted chain of thought instructs a general model to imitate a style of working it saw in training. The intermediate tokens are drawn from the same distribution as ordinary output, because that is what they are. Quality of working is whatever the pretraining and instruction-tuning distributions supply, and the model has no particular incentive to make the working *useful* to itself rather than merely plausible-looking.

A reasoning model has been trained, in the reinforcement-learning stage of Chapter 7's three-stage pipeline, on a reward that depends on the **final answer being correct**, with the intermediate tokens left free. That difference in the objective is the whole difference in behaviour.

Consider what that reward optimises. The model is not rewarded for producing legible working; it is rewarded for producing working that *raises the probability of the correct answer that follows it.* Behaviours that survive such an objective include checking an intermediate result, noticing a contradiction and backtracking, restating a constraint, trying a second approach after the first stalls, and — critically — **spending more tokens on harder problems**. Nobody wrote those behaviours into the reward. They are what the reward selects for, in the same way Chapter 7's reward hacking was what that reward selected for. The mechanism is identical; only the sign of the outcome differs.

This is also why Chapter 7's warning still applies at full force. **A statistical objective optimises exactly what you measured, not what you meant.** A reasoning model rewarded on answer correctness will produce traces that improve answer correctness, which is not the same as traces that are honest, complete, or safe to show a regulator. Chapter 7's reward-hacking experiment — 100% on the in-distribution set, 56.5% on the hard set — is the standing reminder of what a proxy metric does when

you lean on it.

	Prompted chain of thought	Reasoning model
Where it comes from	An instruction in the prompt	The RL objective in training
Length control	Prompt wording, unreliable	A parameter — `effort` on Claude
Adapts to difficulty	Barely	Yes; this is the trained behaviour
Backtracking	Rare; the trace tends to commit	Common; the trace is allowed to be untidy
Where the tokens go	Into the visible response	Into separate `thinking` blocks
Billed as	Output tokens	Output tokens — identically
Can be turned off	Delete the instruction	A parameter, on some models

The last two rows matter more than the others put together. The tokens cost the same either way, and on the current Claude models the amount of them is a request parameter rather than a prompt-engineering exercise. Section 13.10 is the API surface; sections 13.4 to 13.9 are what you should know before you touch it.

13.4 The Three Knobs

There are exactly three ways to spend more compute on one request. Everything in the literature and every product feature is one of them or a combination.

DEFINITION

The three test-time compute knobs

1. Think longer (sequential). One attempt, more intermediate tokens. The extra tokens are serial: each conditions on the last. Claude exposes this as `effort`, and previously as `budget_tokens`.

2. Sample more (parallel). Several independent attempts at the same problem, at a non-zero temperature so they differ. The attempts are independent, so they parallelise perfectly and cost latency only once — but they cost money `n` times.

3. Verify and select. Given several candidates, decide which to return: check them against something, vote among them, or both. This knob generates no tokens of its own and is where almost all of the value is.

Knob 3 is the one people skip, and skipping it is why the summariser pipeline in the introduction paid nine times over for nothing. Knobs 1 and 2 raise the probability that a good answer exists **somewhere**. Knob 3 is what turns that into a good answer being **returned**. Without it you have paid for options you cannot exercise.

Three named strategies combine these knobs, and we will measure all three:

Best-of-n. Sample `n` candidates; return one that a verifier accepts. Needs knobs 2 and 3, and needs a verifier.

Self-consistency (majority voting). Sample `n` candidates; return the most common answer. Needs knobs 2 and 3, but needs **no verifier** — which is exactly why it is so widely reached for, and, as we are

about to measure, exactly why it disappoints.

Extended thinking. One candidate, many more intermediate tokens. Knob 1 only. On a reasoning model this includes a certain amount of internalised knob 3: the trained backtracking behaviour is self-verification that happens inside the trace.

13.5 Coverage and Selection: The Decomposition That Explains Everything

Here is the single most useful idea in the chapter, and it is a definition rather than a result.

> DEFINITION
>
> ### Coverage and selection
>
> Given `n` sampled candidates for a problem:
>
> **Coverage** (also `pass@n`) is the probability that **at least one** candidate is correct. It is a property of the model and the sampling temperature. It is an oracle quantity: computing it requires knowing the right answer, so you can measure it offline on a labelled set and never in production.
>
> **Selection** is the probability that, given a correct candidate is present, your procedure **returns** it. It is a property of your verifier.
>
> Accuracy is the product. `accuracy ≈ coverage × selection`.

Every test-time compute technique moves one of the two factors, and knowing which one tells you what will happen.

- More samples raises **coverage**, with diminishing returns, and does nothing at all for selection.
- A longer thinking budget raises coverage per sample.
- A better verifier raises **selection** and does nothing for coverage.
- A better model raises both, and this is why it is usually the right answer.

Two corollaries follow, and they are the ones to carry into an exam and into a design review.

Test-time compute amplifies; it does not create. If coverage is zero — if the model cannot produce the right answer at any temperature, in any number of attempts — then selection is multiplying by zero and no budget rescues it. This is Chapter 10's structural limit, restated in the currency of this chapter. We measure it in section 13.16, on the five-digit addition that Chapter 10 showed collapsing to exactly 0.0000: coverage over one hundred and twenty-eight samples is also exactly zero, and every strategy returns nothing.

A gap between coverage and accuracy is a verifier problem, not a budget problem. If your measured accuracy is 0.50 and your coverage is 0.89, buying more samples is the wrong purchase. The right answer is already in the pool nine times in ten and you are failing to recognise it. Those are our actual numbers, from section 13.15.

13.6 The Verifier Asymmetry

Now the load-bearing claim.

KEY TAKEAWAYS

The verifier asymmetry

Test-time compute helps most where answers are **cheaply checkable** and least where they are not. The size of the gap is not marginal — on the measurements in this chapter, verified selection beats unverified voting by 37 percentage points on identical samples at identical cost — and it is the first thing to establish about any workload before spending a penny on reasoning.

The reason is section 13.5's product. Sampling raises coverage for everyone. Converting coverage into accuracy requires selection, and selection requires a way of telling right from wrong. Where such a way exists and is cheap, the whole coverage curve is available to you. Where it does not, you are left with proxies for correctness — most commonly agreement among samples — and agreement is a poor proxy for correctness in exactly the cases you care about.

That last point deserves its own sentence, because it is where intuition fails. **A model that is confidently and consistently wrong will vote for the wrong answer by a landslide.** Majority voting measures the mode of the model's output distribution. When the model is well-calibrated and its errors are random noise, the mode is the right answer and voting works. When the model has a systematic bias — it has learned a shortcut, or it is out of distribution, or the question is ambiguous — the mode is the wrong answer and voting converges to it *faster* as you buy more samples. You have spent money to become more certain of a mistake.

It is worth being precise about what makes a good verifier, because "checkable" is doing a lot of work in the claim above.

Verifier	Cost	Soundness	Typical use
Compiler, type-checker, linter	Near zero	High for what it checks	Code generation
Unit or integration test suite	Low	High, bounded by test coverage	Patch and build agents
Schema or grammar validation	Near zero	Total, for structure only	Structured output (Ch 10)
Deterministic recomputation	Low	Total	Arithmetic, transforms, ledgers
Cheap partial check	Near zero	**Incomplete**	Checksums, invariants, parity
Retrieval-grounded citation check	Moderate	Partial — Chapter 8's warning	RAG answers
A second model as judge	**High**	Bounded by the judge	Everything else
A human	Very high	High	Everything else, slowly

Read that table as a ranking of how much test-time compute is worth buying. At the top, a great deal. At the bottom, a second model as judge doubles your bill and caps your selection accuracy at the judge's own reliability — which Chapter 8 taught you to measure with Cohen's kappa before trusting, and which is very often not high enough to justify the arithmetic.

Note the fifth row, because our experiment uses it. A verifier does not have to be complete to be useful, and an incomplete verifier has a measurable ceiling. We use **casting out nines**: a correct sum satisfies `(a + b) mod 9 == answer mod 9`, a number is congruent to its digit sum modulo nine, and so the check costs seven additions and one modulo. It rejects a wrong answer about eight times in nine. It is genuinely cheaper than solving the problem, which is what makes it a verifier rather than a solver — a distinction worth insisting on, since a "verifier" that recomputes the answer from scratch has simply moved the work and proved nothing.

13.7 Visual Analogies

VISUAL ANALOGY

The exam candidate with a mark scheme

Two candidates sit the same paper. Both are allowed to attempt every question ten times on rough paper. The first has the mark scheme; she works all ten, checks each against it, and submits the one that agrees. The second has no mark scheme; he works all ten and submits whichever answer he happened to get most often.

The first candidate's score is limited by whether a correct attempt appears anywhere in her ten — coverage. The second's is limited by whether his most frequent instinct is the right one, which on the questions he misunderstands it emphatically is not. They did identical work. Their scores differ enormously, and the difference is not effort.

Where it breaks. A human with a mark scheme learns from checking; the next question goes better. A model does not. Chapter 1's forward-only inference and Chapter 7's frozen parameters both apply: nothing learned during selection persists past the response. Rejecting sixty-three candidates leaves the model exactly as it was, which is why cost scales linearly with `n` and accuracy does not. The analogy also flatters the mark scheme, which for a human is usually complete; real verifiers, as the table above shows, mostly are not.

VISUAL ANALOGY

Rolling dice against a target

Sampling is rolling dice. Coverage is the chance of hitting your target number at least once in `n` rolls, and it climbs steeply at first and then flattens: the fifth roll adds far less than the second. That is the diminishing-returns curve, and it is the same shape whatever the task.

Verification is being able to *see* the dice. If you can see them, you keep the good roll and the curve is yours. If you cannot, you are asked to guess which roll was good, and your accuracy is whatever your guessing procedure achieves — no matter how many times you rolled.

Where it breaks. Dice rolls are independent and identically distributed; samples from a language model are neither, in a way that matters. They share a prompt, a temperature and a set of weights, so their errors are correlated — the same misreading of the question recurs across all sixty-four. This is precisely why coverage saturates below one instead of climbing towards it, and why majority voting fails on the biased cases: the dice are loaded, all in the same direction, and rolling more of them does not unload them.

VISUAL ANALOGY

A second opinion, and who gets to give it

A patient with an unclear diagnosis can be sent for more tests, or shown to more doctors. More tests is knob 1: deeper investigation of the same case. More doctors is knob 2: independent attempts. Neither helps unless somebody can adjudicate — and the value of adjudication depends entirely on whether there is a definitive test to appeal to.

Where it breaks. Doctors are genuinely independent; they trained in different places and hold different priors. Samples from one model at one temperature are not independent in that sense, and pretending otherwise is the exact error behind over-trusting self-consistency. A closer analogue would be asking one doctor the same question sixty-four times in a slightly different tone of voice — which is, uncomfortably, what parallel sampling is.

13.8 The Economics: The Most Expensive Tokens You Can Buy

Chapter 11's rate card is the whole of the argument here, and it needs only one additional fact, which the platform documentation states directly: **thinking tokens are billed as output tokens**, whether or not the thinking text is returned to you.

That single sentence has large consequences, because on every current Claude model output is priced at five times input.

Model	Input $/MTok	Output $/MTok	Ratio
Claude Fable 5	10	50	5×
Claude Opus 5	5	25	5×
Claude Sonnet 5	2	10	5×
Claude Haiku 4.5	1	5	5×

Sonnet 5 at $2/$10 through 31 August 2026, then $3/$15, according to the official pricing documentation at the time of writing.

Take a concrete request on Claude Opus 5: 2,000 input tokens, 500 output tokens.

```
without thinking
  input    2,000 x $5  / 1M  =  $0.0100
  output     500 x $25 / 1M  =  $0.0125
                        total =  $0.0225
with an 8,000-token thinking trace
  input    2,000 x $5  / 1M  =  $0.0100
  thinking 8,000 x $25 / 1M  =  $0.2000     <- billed as output
  output     500 x $25 / 1M  =  $0.0125
                        total =  $0.2225     9.9x
```

Nine point nine times, for one parameter. At the introduction's million requests a month that is $22,500 becoming $222,500, which is the summariser team's bill and now you know where it came from. Three further facts sharpen the picture.

Thinking is not cacheable. Chapter 11's cache multipliers — 1.25× for a five-minute write, 2× for an hour, 0.1× for a read — apply to *input*. Thinking tokens are generated fresh on every request. There is no prefix to hit. The one lever that made Chapter 11's prefix-heavy workload cheap does not reach this line item at all.

Changing the thinking configuration invalidates the cache you do have. The documentation is explicit: the thinking configuration and the resolved effort level are rendered into the prompt, so switching between `adaptive`, `enabled` and `disabled`, changing `budget_tokens`, or changing the effort value all invalidate cache breakpoints. Chapter 11's misplaced-prefix failure cost +20.6% against no caching at all; varying effort inside a cached conversation is the same failure with a different cause. Pick a level and hold it for the life of the conversation.

Latency moves with it, and by more than you expect. Chapter 11's model was `latency ≈ prefill + output_tokens × per_token_decode`, with decode at 82–99% of wall clock. Going from 500 output tokens to 8,500 is a seventeen-fold increase in the term that dominates. The running thread stands: **decode dominates latency, and generating fewer tokens is the only lever that improves latency, throughput and cost together.** Reasoning is that lever pulled hard in the wrong direction, deliberately, in exchange for accuracy. That can be an excellent trade. It is never a free one.

Parallel sampling prices differently, and better than people expect, because the input amortises. Best-of-8 on the same request, with prompt caching:

```
cache write  2,000 x $5 x 1.25 / 1M  =  $0.0125
cache reads  7 x 2,000 x $5 x 0.1 / 1M =  $0.0070
output       8 x 500 x $25 / 1M       =  $0.1000
                              total  =  $0.1195     5.3x
```

Five point three times for eight attempts, not eight times — because seven of the eight prompts are cache reads at a tenth of the input rate. Chapter 11's caching arithmetic is what makes parallel test-time compute affordable, and it is the reason best-of-n is often the cheaper way to buy accuracy than a long thinking trace. It is also why the **Batch API's 50% discount** on both input and output belongs in any serious best-of-n design: sampling is embarrassingly parallel and rarely latency-critical.

13.9 Architecture: Where This Sits

In a production system, test-time compute is not a model setting. It is a **routing decision plus a verification stage**, and the model setting is a consequence of both.

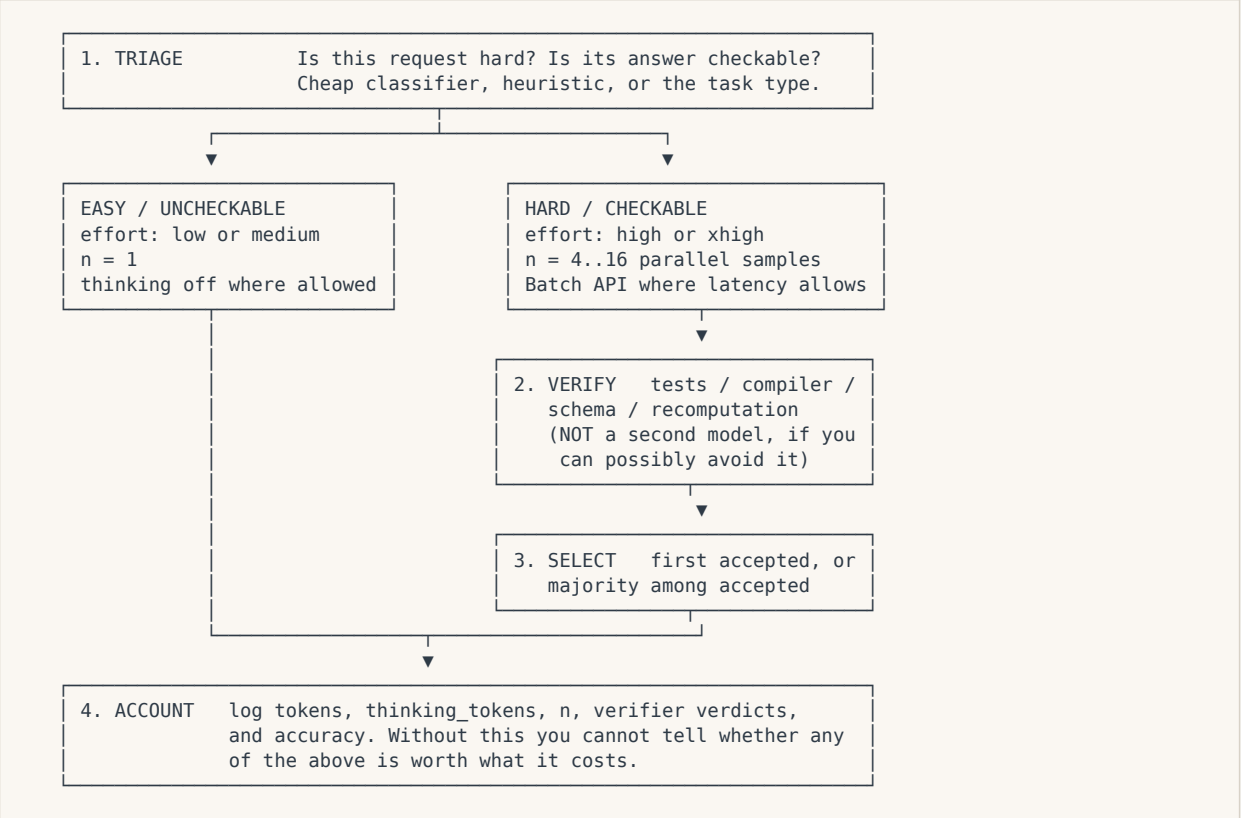


Figure 13.1 — Test-time compute as an architectural stage, not a parameter.

Four observations about that diagram, each of which is a running thread cashed out.

The model is a component, not the system. Verification and selection are your code. The model proposes; your system disposes. Chapter 10's compensating architecture said the fix for a structural limit is never a better prompt; the fix for a *selection* problem is never a better prompt either.

Triage comes before optimisation. Chapter 10's four-way triage — information, exact execution, external state, checkability, with prompting as the fall-through — extends cleanly here. Reasoning is worth buying for a problem that is genuinely hard *and* whose answers you can check. For anything else it is a tax.

Verification is where the leverage is, and it is usually the cheapest stage. A test suite costs milliseconds. Sixteen samples from Opus 5 cost dollars per thousand requests. Spending engineering effort on the verifier has a better return than spending tokens on the model, almost always.

Accounting is not optional. Chapter 12's discipline applies: measure the accuracy with an interval, measure the tokens, and compare against the same workload with the knobs off. The introduction's summariser team had the bill and not the accuracy comparison, which is why it took a week to discover they had bought nothing.

13.10 The Claude Surface: Adaptive and Extended Thinking

Everything in this section was verified against the platform documentation at `platform.claude.com/docs/en/build-with-claude/thinking`, `.../extended-thinking`, `.../effort` and `.../task-budgets` at the time of writing. Chapter 18 covers the API mechanics in depth — streaming

events, block handling, the SDK surface. This section is what you need to reason about the *choice*.

There are two thinking modes, and which one you get is a property of the model.

DEFINITION

Adaptive thinking

`thinking: {"type": "adaptive"}`. **Claude decides whether and how much to think on each request.** Depth is steered by `effort` rather than a token budget. On simple inputs at lower effort the model may skip thinking entirely, producing no thinking block at all. This is the current mode and the one to design for.

DEFINITION

Extended thinking (legacy)

`thinking: {"type": "enabled", "budget_tokens": N}`. **You set a fixed target budget and Claude thinks on every request.** Minimum 1,024 tokens; must be less than `max_tokens`, except under interleaved thinking. The documentation marks it deprecated on the 4.6 generation, and **Claude 4.7 and later reject it with a 400 error**. It remains the only mode on Claude 4.5 and earlier models that think at all, including Claude Haiku 4.5.

The per-model picture, from the documentation at the time of writing:

Model	Mode	Default state	Display default
Claude Fable 5	Adaptive	On; cannot be disabled	`omitted`
Claude Opus 5	Adaptive	On; disable allowed at `high` and below	`omitted`
Claude Sonnet 5	Adaptive	On; can be disabled	`omitted`
Claude Opus 4.8 / 4.7	Adaptive	Off until you set it	`omitted`
Claude Opus 4.6 / Sonnet 4.6	Adaptive (extended deprecated)	Off until you set it	`summarized`
Claude Haiku 4.5	Extended only	Off until you set it	`summarized`

Three details on that table carry real design weight.

Thinking is on by default on the 5-series. Opus 5, Sonnet 5 and Fable 5 think unless you tell them not to. If you are migrating a cost model from an older generation, this is the line item that surprises you. Opus 5 accepts `thinking: {"type": "disabled"}` at effort `high` or below; at `xhigh` or `max` it returns a 400 error, because at those levels thinking is the point. Fable 5 and Mythos 5 reject `disabled` outright.

`display` **defaults to** `omitted` **on the newest models, and omitting is not free.** With `display: "omitted"` the thinking block comes back with an empty `thinking` field and only its `signature`. The documented benefit is **faster time-to-first-text-token when streaming**, because the server skips streaming the thinking tokens. The documentation is equally explicit about what it is not: *you are still charged for the full thinking tokens. Omitting reduces latency, not cost.*

What you can see is a summary, not the raw trace. With `display: "summarized"` the text you receive is a summary of the reasoning, produced by a **different model** from the one you targeted, and you are billed for the full thinking tokens rather than the summary you can read. No setting returns the raw chain of thought. Practical consequence: the visible token count will not match the billed one, and you should reconcile against `usage.output_tokens_details.thinking_tokens`, which reports how many billed output tokens were internal reasoning. When streaming, that breakdown arrives only on the final `message_delta` event.

13.11 Effort: The Knob You Will Actually Use

`output_config: {"effort": ...}` is the primary depth control wherever adaptive thinking is available. It takes five values.

Level	What it does	Where it is supported
`max`	No constraint on token spend	Fable 5, Mythos 5, Opus 5, Opus 4.8, Opus 4.7, Opus 4.6, Sonnet 5, Sonnet 4.6
`xhigh`	Extended, for long-horizon agentic work	Fable 5, Mythos 5, Opus 5, Opus 4.8, Opus 4.7, Sonnet 5
`high`	**The API default.** Identical to omitting the parameter	All models that support effort
`medium`	Balanced; moderate token savings	All models that support effort
`low`	Most efficient; some capability reduction	All models that support effort

Four things about `effort` that are easy to get wrong and are therefore worth memorising.

The default is `high`, and `high` is expensive. Setting `effort: "high"` produces exactly the same behaviour as omitting it. If you have never set the parameter, you are running at high effort. That is the introduction's summariser bill in one sentence.

It governs all tokens, not only thinking. The documentation is specific: effort affects text responses, tool calls and function arguments, and thinking. Lower effort means fewer tool calls, terser confirmations, less preamble. It works whether or not thinking is active. This makes it a broader instrument than a thinking budget ever was.

It is a behavioural signal, not a cap. At lower effort Claude still thinks on sufficiently difficult problems; it simply thinks less. If you need a hard ceiling, `max_tokens` is the strict limit and effort is not.

Do not pass `adaptive` as an effort value. `adaptive` is a thinking mode; the effort scale is the five levels above. The documentation calls this out explicitly, which suggests it is a common enough mistake to be exam material.

The documented model-specific guidance is worth internalising as a starting point rather than a rule: on Opus 5, start at the `high` default and step *up* to `xhigh` for demanding coding and agentic work, using `low` and `medium` liberally wherever your evaluations show quality holds; on Opus 4.7 and 4.8, start at `xhigh` for coding and agentic use; on Sonnet 4.6, the documentation recommends `medium` as the practical default

and warns that leaving it unset produces unexpected latency. The through-line in all of it is that effort settings **do not transfer between models**. The documentation says so directly for Opus 5: if you carried settings over from an earlier model, run a fresh sweep rather than reusing them. Chapter 12's rule about benchmarks applies to parameters too — measure on your own task.

13.12 Task Budgets, Thinking Blocks, and the Context Window

Three more surfaces, each of which changes an architecture decision.

Task budgets (beta). `output_config.task_budget` gives Claude an advisory token allowance for a whole agentic loop — thinking, tool calls, tool results and output — with a countdown the model can see and pace itself against. It is in beta behind the `task-budgets-2026-03-13` header on Opus 5, Fable 5, Mythos 5, Opus 4.8 and Opus 4.7; it is **not supported on Sonnet 5**, nor on Claude Code or Cowork surfaces. The minimum accepted total is **20,000 tokens** and smaller values return a 400. Three properties matter architecturally: the budget is **advisory, not enforced** — `max_tokens` remains the hard cap; the countdown is **visible only to the model**, with no field in the response `usage` object; and a budget that is obviously too small for the work can produce **refusal-like behaviour**, with the model declining or scoping down rather than starting something it cannot finish. If you see unexplained early stops after adding a budget, raise the budget before debugging anything else. Where effort tunes depth, task budgets tune breadth; they compose.

Thinking blocks are signed and must be returned unmodified. Every thinking block carries a `signature` field: an encrypted copy of the full reasoning that the API uses to verify the block was generated by Claude. Within a tool-use turn you **must** pass thinking blocks back complete and unmodified alongside the `tool_use` block they accompanied; modified blocks are rejected with a 400. Across turns it is recommended. The API filters what it needs and bills input tokens only for the blocks actually shown to the model. `redacted_thinking` blocks — returned when reasoning is safety-redacted — must be round-tripped the same way, which means code that filters on `block.type == "thinking"` alone silently breaks the protocol. And when you switch models mid-conversation, strip prior thinking blocks: they are tied to the model that produced them, and other models ignore them while still charging you for the input.

Thinking counts against the context window. Current-turn thinking always counts toward `max_tokens`, is billed as output, and occupies window space. What happens to *prior*-turn thinking is per-model: Opus 4.5 and later Opus models, Sonnet 4.6 and later Sonnet models, Fable 5 and Mythos 5 **keep all prior turns**, billing them as input; earlier Opus and Sonnet models and every Haiku through 4.5 **keep only the last turn**, stripping the rest automatically. On keep-all models, budget your window as though thinking were ordinary conversation history, because it is — and a long agentic session accumulates it. This is the first appearance of a problem Part 10 is entirely about; Chapter 54 covers compaction and the `clear_thinking_20251015` context-editing strategy that reclaims the space.

Interleaved thinking. With adaptive thinking, Claude reasons *between* tool calls automatically, with no beta header, on every model that supports adaptive mode. Claude Haiku 4.5 does not support interleaved thinking at all. Under manual extended thinking it requires the `interleaved-thinking-2025-05-14` header, and there the `budget_tokens` allowance spans the whole assistant turn rather than one response — the one case where it may exceed `max_tokens`. Chapter 35 returns to what this means for tool chaining and Chapter 50 to what it means for orchestrator–subagent designs.

13.13 Four Compatibility Facts Worth Memorising

Short section, high exam density. All four are documented behaviours, not folklore.

1. **Non-default `temperature`, `top_p` or `top_k` return a 400 error** on Fable 5, Mythos 5, Mythos Preview, Opus 5, Opus 4.8, Opus 4.7 and Sonnet 5 — on every request, whether or not thinking is used. On older models the restriction applied only while thinking was on. The practical consequence for this chapter is significant: **you cannot tune sampling temperature for parallel sampling on the current models.** Diversity across samples has to come from the prompt, from the model's own stochasticity, or from a lower-effort model used as the sampler.
2. **You cannot prefill the assistant response while thinking is on.** Forced tool use (`tool_choice: "any"` or a named tool) is incompatible with *manual* extended thinking but works fine with adaptive.
3. **Above roughly 32,000 thinking tokens, use the Batch API.** Long-running requests hit system timeouts and open-connection limits. Relatedly, the SDKs require streaming when `max_tokens` exceeds 21,333 — a client-side guard, not an API restriction.
4. **Thinking-heavy work often outlives the five-minute cache.** The documentation recommends the one-hour cache duration for multistep thinking workflows. Chapter 11 priced that at a 2× write multiplier against 1.25×; if it converts a miss into a hit, it pays for itself immediately.

One more, specific to Fable 5 and worth flagging because it is a governance question disguised as an API detail: on Fable 5, a request that tries to elicit the model's internal reasoning as part of the response text can be **refused**, with `stop_details.category: "reasoning_extraction"`. The supported route to visibility is reading the `thinking` blocks, not prompting for a narrative of the reasoning. Section 13.22 returns to why that distinction is a feature rather than an obstruction.

13.14 Examples: Four Workloads, Four Answers

Before the code, four concrete decisions, so the framework has something to bite on. Each names the checker, because that is the first question.

A build-fixing agent. Failing test suite in, patch out. Checker: the test suite, which costs seconds and is close to sound. Coverage is decent because the model can usually produce a working patch given enough attempts. **Buy heavily.** High or `xhigh` effort, four to sixteen parallel samples, run through the tests, return the first that passes. This is the introduction's second pipeline and its 38%-to-71% jump.

A support-ticket summariser. Conversation in, paragraph out. Checker: none that is cheap. Quality is a rubric and a human. Coverage is high — the model produces an acceptable summary on the first attempt almost every time — so there is little headroom to buy. **Buy nothing.** `effort: "low"` or `medium`, thinking disabled where the model allows it, `n = 1`. This is the introduction's first pipeline and its nine-fold bill for nothing.

A financial-reconciliation assistant. Two ledgers in, discrepancy list out. Checker: recomputation, which is total and nearly free. But note Chapter 10's triage first: if the task is exact arithmetic over structured data, the right answer is a tool, not a reasoning trace. **Buy modestly, and only after the tool.** Give the model code execution or a calculator, use `medium` effort for the analysis around it, and verify the totals deterministically.

A legal-clause classifier. Contract clause in, category out. Checker: none, unless you have labels — and if you have labels you have training data, which is a different chapter. Coverage may be high but selection is unavailable, so self-consistency is the only option and section 13.15 is about to show what that is worth. **Buy nothing; buy a better model or a better prompt instead**, and measure with Chapter 8's intervals. Note the specific trap: a classifier over an ambiguous taxonomy is exactly the case where the model is *consistently* wrong, so majority voting will report high agreement and low accuracy. Agreement is not correctness, and a confidence signal built from agreement will be confidently wrong.

13.15 Code Walkthrough: A Harness That Measures Instead of Asserting

Everything claimed in sections 13.5 and 13.6 is testable, so we test it. The harness lives in `ch13/ttc.py` and reuses Chapter 10's addition task, for three reasons: the ground truth is deterministic, a genuinely incomplete checker exists in casting out nines, and Chapter 10 already established that the task has a hard structural cliff at five digits — which gives us a free demonstration of what happens when coverage is zero.

The design goal is that all four strategies see **the same samples**. If best-of-n and self-consistency draw independently, any difference between them is confounded with sampling noise. Drawing once and selecting four ways makes the comparison exact and the token column honest.

Sampling. The Chapter 10 network emits a per-position softmax over ten digits and takes the argmax. To get candidates we sample from that softmax at a temperature instead.

```
def probs(net, X, temperature=1.0):
    logits = net.forward(X)[2] / temperature
    e = np.exp(logits - logits.max(-1, keepdims=True))
    return e / e.sum(-1, keepdims=True)
def sample(net, X, n, rng, temperature=1.0):
    p = probs(net, X, temperature)                    # (b, D, 10)
    b, D, V = p.shape
    flat = p.reshape(-1, V)
    cdf = flat.cumsum(-1)
    cdf /= cdf[:, -1:]
    u = rng.random((n, flat.shape[0], 1))
    idx = (u > cdf[None, :, :]).sum(-1)
    return idx.reshape(n, b, D)
```

Line by line. `logits / temperature` is exactly Chapter 3's definition: dividing before the softmax flattens the distribution as temperature rises and sharpens it as temperature falls. Subtracting the row max before exponentiating is the standard numerical-stability trick and changes nothing mathematically. The `cdf`/`u` construction is inverse-transform sampling done in one vectorised shot: `(u > cdf).sum(-1)` counts how many cumulative buckets the uniform draw cleared, which is the sampled index. The result is `(n, b, D)` — `n` independent candidate answers for each of `b` problems.

The verifier. Casting out nines, which is the whole point of the exercise.

```
def casting_out_nines(a, b, ans):
    return (to_int(ans) % 9) == ((a + b) % 9)
```

Two properties make this a fair stand-in for a production verifier. It is **cheap** — one modulo per operand, no reconstruction of the answer. And it is **incomplete**: a wrong answer congruent to the true sum modulo nine passes. If the model's errors were uniformly distributed, that would be one in nine, or 0.1111. Our measured false-accept rate across the sweep is **0.0742**, meaningfully lower, because the model's errors are not uniform — it tends to be wrong in the low-order digits, which the modulo is comparatively good at catching. That is a small, honest result in its own right: verifier soundness is a property of the *pair*, checker and error distribution, not of the checker alone.

For contrast the module also provides `oracle`, which is exact and therefore not a verifier at all, and `null_verifier`, which accepts everything and models the uncheckable regime. Running the same samples through all three isolates the verifier as a variable.

Selection. Three procedures, each a few lines.

```
def select_first_accepted(cands, accept):
    n, b, D = cands.shape
    any_ok = accept.any(0)
    first = np.where(any_ok, accept.argmax(0), 0)
    return cands[first, np.arange(b)]
```

`accept.argmax(0)` finds the first `True` along the sample axis; where nothing was accepted we fall back to candidate zero rather than returning nothing, because a production system has to answer. Note what this makes true by construction: with `null_verifier`, every candidate is accepted, `argmax` returns index 0, and best-of-n degenerates exactly to a single sample. There is a test asserting that, because it is the formal statement of the verifier asymmetry.

`select_majority` votes on the whole answer, breaking ties by **first occurrence**. That choice matters and it is deliberately conservative: it makes a tie degrade to `single` rather than to chance, so any deficit we report for voting is real and not an artefact of tie-breaking.

Cost. One function, taken straight from Chapter 12.

```
def compute_units(params, n_samples, tokens_per_sample=ANSWER_TOKENS):
    return 2 * params * tokens_per_sample * n_samples
```

Chapter 12 gave training compute as `C ≈ 6ND`, of which two are the forward pass. Generating one token therefore costs about `2N` FLOPs, and test-time compute is that times the tokens you choose to generate. This is the unit that lets us put "more samples from a small model" and "one sample from a bigger model" on the same axis in section 13.16, which is the only way to answer the crossover question honestly.

The suite in `ch13/test_ttc.py` is nineteen tests and runs in four seconds. The ones that matter are not the unit tests on the primitives but the four invariants: coverage bounds every strategy from above; coverage is monotone in budget; a perfect verifier reaches coverage **exactly**; and with no verifier, best-of-n **equals** `single`. Those four are the chapter's argument, written as assertions.

13.16 Mini Project: The Test-Time Compute Ledger

Goal. Take a task with a checkable answer, spend inference compute on it five different ways across a budget grid, and produce a ledger that says — with intervals — what each purchase bought and what it

cost. Then run the identical harness on a task where the answer cannot be checked, and on one where the model cannot produce the answer at all, and let both fail.

13.16.1 Architecture

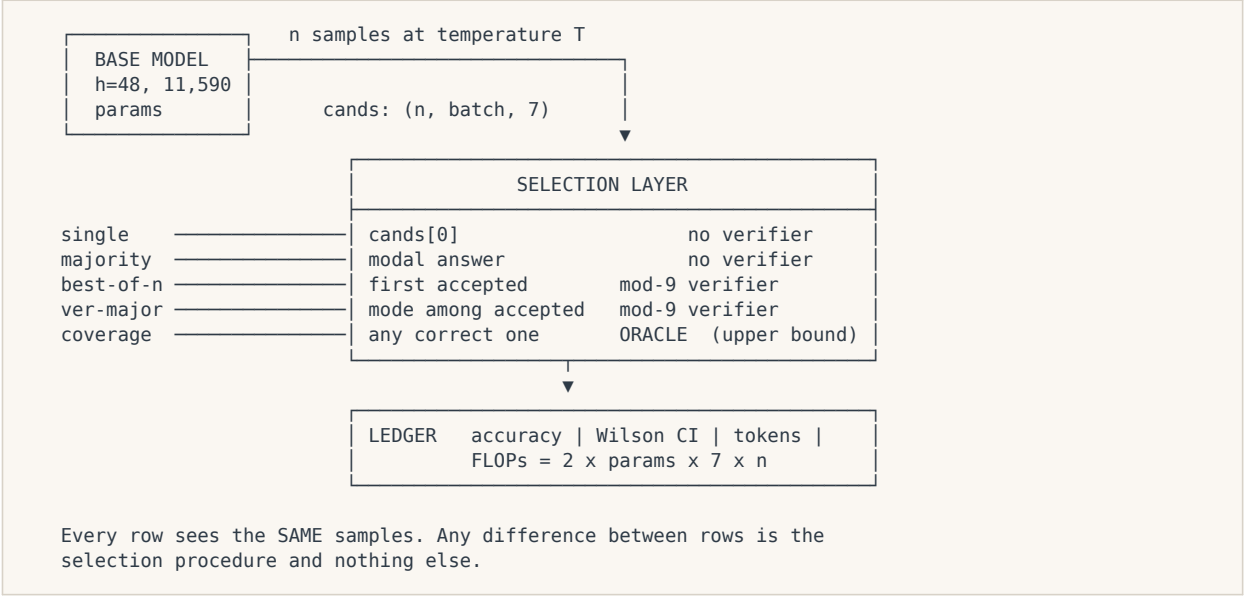


Figure 13.2 — The ledger. One set of samples, five ways of spending it.

13.16.2 Folder structure

```
ch13/
  arith.py           copied from ch10 — the task and the network
  ttc.py             sampling, verifiers, selection, the sweep, cost
  train_models.py    trains net_small.npz and net_large.npz (resumable)
  report.py          the five panels below
  test_ttc.py        19 tests
  net_small.npz      h=48,  11,590 params
  net_large.npz      h=96,  27,718 params
```

13.16.3 The base models, and why not Chapter 10's

Chapter 10's network scores 0.9967 exact on in-distribution addition. A model that is already right has no headroom, and a curve with no headroom teaches nothing. We instead train two sizes lifted straight from Chapter 12's sweep, so the crossover comparison is against a measured point rather than a guess:

```
base models on n=20,000 held-out 4-digit sums
  small  h=48  11,590 params   greedy exact 0.1308
  large  h=96  27,718 params   greedy exact 0.9047
```

Both reproduce Chapter 12's published figures to four decimal places, which is the first thing to check when reusing an earlier chapter's machinery.

13.16.4 Panel A — no diversity, no test-time compute

```
A  TEMPERATURE: no diversity, no test-time compute (n=16)
        T   single  majority  best-of-n  coverage
      0.0   0.1308    0.1308     0.1308    0.1308
```

```
      0.3   0.1024    0.1232     0.4606    0.5534
      0.5   0.0818    0.1138     0.4426    0.5815
      0.7   0.0682    0.1017     0.3987    0.5596
      1.0   0.0534    0.0835     0.3404    0.5069
      1.2   0.0445    0.0712     0.2928    0.4618
```

The first row is the one to sit with. At temperature zero all sixteen samples are **identical**. Sixteen times the tokens, sixteen times the bill, and the four columns agree because there is nothing to choose between. Diversity is not a nice-to-have for parallel test-time compute; it is the precondition. Any system that samples repeatedly at temperature zero is burning money by construction.

The rest of the table is the diversity/quality trade in miniature. Raising temperature makes each individual sample worse — `single` falls monotonically from 0.1024 to 0.0445 — while coverage rises to a peak at T=0.5 and then falls again. Too little diversity and the samples are copies; too much and they are noise. We take T=0.5 as the operating point for everything that follows, chosen on coverage before looking at any accuracy number.

> WARNING
>
> **A limit of this experiment, stated up front**
>
> Section 13.13 established that non-default `temperature`, `top_p` and `top_k` return a **400 error** on the current Claude models. The temperature sweep above is therefore something you can do with your own network and cannot do with Opus 5 or Sonnet 5. It is included because it isolates the mechanism — no diversity, no gain — and that mechanism still governs, but on the Claude API your diversity comes from the model's own stochasticity and from prompt variation, not from a temperature parameter. Do not read Panel A as tuning advice for a production Claude workload.

13.16.5 Panel B — the budget curve

Twenty thousand held-out problems, T=0.5, budgets from one sample to one hundred and twenty-eight.

```
B  BUDGET CURVE, VERIFIER AVAILABLE (small model, T=0.5)
       n  tokens   single  majority  best-of-n  ver-maj  coverage
       1       7   0.0818    0.0818     0.0818   0.0818    0.0818
       2      14   0.0835    0.0835     0.1495   0.1495    0.1537
       4      28   0.0824    0.0912     0.2449   0.2457    0.2663
       8      56   0.0858    0.1066     0.3535   0.3583    0.4139
      16     112   0.0871    0.1153     0.4385   0.4567    0.5793
      32     224   0.0808    0.1155     0.4839   0.5250    0.7241
      64     448   0.0833    0.1245     0.4980   0.5694    0.8294
     128     896   0.0824    0.1260     0.5011   0.5881    0.8917
    best-of-128 95% CI [0.4942, 0.5080]
    verifier false-accept rate 0.0742 (1/9 = 0.1111 if errors were uniform)
```

Four readings.

Test-time compute is real. Best-of-8 reaches 0.3535 against a single greedy sample's 0.1308 — a 2.7× gain for eight times the tokens. That is a genuine purchase, on a model whose weights never changed.

Returns diminish, sharply. From n=1 to n=8, best-of-n gains 0.2717. From n=64 to n=128 — doubling again, at eight times the absolute token cost of the first jump — it gains **0.0031**. Chapter 8's intervals settle whether that last gain is even real: the 95% Wilson interval on best-of-128 is [0.4942, 0.5080],

which contains best-of-64's point estimate of 0.4980. **The final doubling is not an established improvement.** Four hundred and forty-eight extra tokens per problem bought nothing we can demonstrate.

The curves part company, and that is the finding. Best-of-n flattens after n=32. Coverage does not: it climbs from 0.7241 to 0.8917 over the same range. More sampling keeps landing on the right answer; the verifier stops being able to find it. Section 13.16.6 quantifies exactly whose fault that is.

`verified_majority` **beats** `best_of_n`**, and it should.** Voting *among the accepted candidates* reaches 0.5881 against 0.5011 — nearly nine points, for zero extra tokens. The mod-9 check admits about 7.4% of wrong answers, but wrong answers that pass are individually unlikely to recur, whereas the right answer recurs. Pruning with the verifier and then voting inside the survivors combines both signals. If you take one implementation detail from this chapter, take that one: **verify then vote is strictly better than verify then take the first**, and costs nothing.

13.16.6 Panel B2 — whose ceiling is it?

Run identically, but hand the harness a perfect verifier.

```
B2 WHOSE CEILING IS IT? Same samples, a PERFECT verifier
       n    mod-9   oracle  coverage  lost to verifier
       1   0.0818   0.0818    0.0818            0.0000
       2   0.1495   0.1537    0.1537            0.0042
       4   0.2449   0.2663    0.2663            0.0214
       8   0.3535   0.4139    0.4139            0.0604
      16   0.4385   0.5793    0.5793            0.1408
      32   0.4839   0.7241    0.7241            0.2402
      64   0.4980   0.8294    0.8294            0.3315
     128   0.5011   0.8917    0.8917            0.3906
```

With a perfect verifier, best-of-n **is** coverage — the middle and right columns are identical to every decimal place, which is not a coincidence but an identity, and there is a test asserting it. Selection becomes free.

The last column is therefore a clean measurement of one thing: **accuracy the sampling already earned and the checker threw away.** At n=128 that is 0.3906 — thirty-nine points of correct answers, sitting in the candidate pool, unrecognised. And notice the shape. The loss grows monotonically with budget, because every extra sample adds another chance for a wrong-but-congruent answer to be encountered *before* the right one. **An imperfect verifier gets worse as you sample more.** That is the mathematical statement of why buying budget cannot fix a selection problem, and it is the single most useful diagnostic in the chapter: measure coverage on a labelled set, compare it to your live accuracy, and the gap tells you which stage to invest in.

13.16.7 Panel C — the asymmetry, priced

Same samples again, verifier removed entirely.

```
C  THE SAME SAMPLES, VERIFIER REMOVED (the asymmetry)
       n  tokens  checkable  uncheckable      gap
       1       7     0.0818       0.0818   0.0000
       2      14     0.1495       0.0835   0.0660
       4      28     0.2449       0.0912   0.1537
       8      56     0.3535       0.1066   0.2469
```

```
      16     112     0.4385       0.1153   0.3232
      32     224     0.4839       0.1155   0.3684
      64     448     0.4980       0.1245   0.3734
     128     896     0.5011       0.1260   0.3751
```

At sixty-four samples: **0.4980 if you can check, 0.1245 if you cannot.** Identical model, identical samples, identical token bill. The only difference is whether a checker exists, and it is worth 37.3 percentage points.

And now the result we did not want.

> WARNING
>
> **A negative result, published**
>
> Self-consistency over sixty-four samples scores **0.1245**. A single greedy sample scores **0.1308**. Majority voting lost to doing nothing at all, at sixty-four times the cost — and it loses at every budget we measured, from n=2 to n=128.
>
> The mechanism is the one section 13.6 predicted. Voting returns the mode of the sampled distribution. Greedy decoding returns the mode of the *per-digit* distribution, which for a seven-digit answer is a sharper estimate of the same thing from one sample. Sampling at T=0.5 and voting is a noisy, expensive approximation of what argmax computes exactly and for free. The gap is small — 0.0063, and the intervals are close — but the direction held across every budget, and the honest summary is not "voting helps a little" but "on this task, at this temperature, voting did not beat greedy and we cannot recommend it".
>
> Chapter 12 published a hypothesis that did not survive its own confidence intervals; Chapter 10 did the same. This is the third. The discipline applies to the author, and self-consistency is a technique we expected to look better than it does.

The generalisable form of that failure: **self-consistency helps when the model's errors are random and hurts when they are systematic.** Random errors scatter and the correct answer wins the vote. Systematic errors concentrate, and voting simply finds the systematic error faster. Before deploying majority voting, measure whether your errors scatter. If your model is confidently wrong in a consistent direction, voting will report high agreement — which you will be tempted to read as confidence — and low accuracy.

13.16.8 Panel D — amplification is not creation

Five-digit operands. Chapter 10 measured this network at exactly 0.0000 exact-match on them, with per-digit accuracy still 0.7764, because it learned four columns rather than an operation.

```
D  OUT OF COVERAGE: 5-digit operands, never seen in training
    greedy exact 0.0000
       n  best-of-n  majority  coverage
       1     0.0000    0.0000    0.0000
       8     0.0000    0.0000    0.0000
      64     0.0000    0.0000    0.0000
     128     0.0000    0.0000    0.0000
```

Every cell is zero, including coverage over one hundred and twenty-eight samples. Not small: **zero**. The correct answer never appeared once, in 2.56 million sampled digits, so there was nothing for any selection procedure to select.

This is Chapter 10's structural limit expressed in this chapter's currency, and it is the cleanest available demonstration that `accuracy = coverage × selection` has a zero in it. Test-time compute amplifies a capability the model has. It does not manufacture one it lacks. Chapter 10's escalation ladder still applies unchanged: for a structural limit the fix is a tool, not a budget — and `tool_add(a, b)` gets 1.0000 on twelve-digit operands with no training, no sampling and no verifier.

13.16.9 Panel E — samples or parameters?

The crossover the whole chapter has been building to. Both models priced in the same unit.

```
E  CROSSOVER: spend on samples, or spend on parameters?
    large model, 1 greedy sample : 0.9047 exact, 388,052 FLOPs
       n  best-of-n        FLOPs  vs large
       1     0.0818      162,260     0.42x
       2     0.1495      324,520     0.84x
       4     0.2449      649,040     1.67x
       8     0.3535    1,298,080     3.35x
      16     0.4385    2,596,160     6.69x
      32     0.4839    5,192,320    13.38x
      64     0.4980   10,384,640    26.76x
     128     0.5011   20,769,280    53.52x
```

At roughly equal compute — small best-of-2 at 0.84× the large model's cost — the scores are **0.1495 against 0.9047**. At fifty-three times the compute the small model reaches 0.5011 and is still not close. On this task, at these sizes, **there is no budget at which spending inference compute on the small model matches simply using the larger one.** The crossover does not exist.

That is a strong claim and it needs its boundaries stated honestly. Our two models differ by 2.4× in parameters and sit on either side of Chapter 12's metric knee, which is the steepest part of the curve — the least favourable place possible for test-time compute to compete. On a flatter part of the curve, or with a much larger sampling model, or with a better verifier than a modulo check, the crossover can and does exist; the published literature reports regimes where it clearly does. What our measurement establishes is narrower and still useful: **the crossover is a measurement, not a law, and on a task where the smaller model is genuinely under-capable it will not be there.** Buying compute cannot substitute for capability you do not have.

The large model's own curve is the constructive half of the same table:

```
       n  best-of-n  coverage        FLOPs
       1     0.7709    0.7709      388,052
       4     0.9468    0.9648    1,552,208
       8     0.9612    0.9871    3,104,416
      32     0.9710    0.9982   12,417,664
      64     0.9708    0.9994   24,835,328
```

Best-of-8 reaches 0.9612 against greedy's 0.9047 — five and a half points, for eight times the tokens, on a model that was already good. That is test-time compute working exactly as advertised. And the same ceiling reappears: coverage at n=32 is 0.9982 while the achieved score is 0.9710. Even here, with a strong model, **two and a half points of correct answers are being discarded by the verifier**, not missed by the sampler.

KEY TAKEAWAYS

The ordering rule this project establishes

Pick the model first, then buy samples, then improve the verifier — and check after each step whether the next one is still worth it. Every one of the five panels above is a measurement of what happens when that order is violated.

13.16.10 Evaluation and deployment

Run `python3 train_models.py` (about thirteen seconds for both), then `python3 report.py` (about twenty-seven seconds) to reproduce every number above, then `python3 -m pytest` for the nineteen tests. The checkpoints are weights-only and small enough to commit.

To deploy this pattern against a real model, three substitutions and one addition. Replace `sample` with `n` parallel API calls sharing a cached prompt prefix — Chapter 11's arithmetic in section 13.8 is what makes that affordable, and the Batch API's 50% discount is what makes it cheap. Replace `casting_out_nines` with your real checker: the test suite, the schema validator, the compiler. Replace `compute_units` with the actual rate card. And add the accounting stage from Figure 13.1, logging `usage.output_tokens_details.thinking_tokens` alongside `n` and the verifier verdicts, because the ledger is the deliverable — the point of the exercise is not the harness, it is being able to answer "what did that buy" with a number and an interval.

13.17 Hands-on Labs

Lab 1 — Establish your own baseline. Take any task you own with a checkable answer. Measure single-sample accuracy on at least 400 examples and print the Wilson interval. *Success criterion:* you can state your baseline as a point estimate and an interval, and you know how many examples would be needed to resolve a five-point improvement.

Lab 2 — Measure coverage. Sample n=16 at whatever diversity you can obtain and compute `pass@16` against ground truth. *Success criterion:* you have a coverage number, and you can say whether your accuracy gap is a coverage problem or a selection problem.

Lab 3 — Price the trace. For one representative request, run it with thinking at your current settings and read `usage.output_tokens_details.thinking_tokens`. Compute the cost with and without, using Chapter 11's rate card. *Success criterion:* a monthly figure for what thinking is costing you at current volume, and the multiple against thinking off.

Lab 4 — Sweep effort. Run your evaluation set at `low`, `medium` and `high`. Record accuracy with intervals, mean output tokens and p50 latency at each. *Success criterion:* you can name the lowest effort level at which quality is not established to have degraded, and say what dropping to it would save.

Lab 5 — Break majority voting on purpose. Construct or find a set of inputs where your model is consistently wrong — an ambiguous category, an out-of-distribution slice. Run self-consistency at n=16 and record both accuracy and the winning margin. *Success criterion:* you have demonstrated high agreement with low accuracy on your own system, and can explain to a colleague why agreement is not a confidence signal.

13.18 Lab Solutions

Lab 1. Reuse `ttc.wilson(k, n)`. On 400 examples at 70% accuracy the interval is roughly ±4.5 points, so a five-point improvement is at the edge of resolvable; Chapter 12's sizing arithmetic gives about 1,700 examples per arm for a two-point gap at 90%. If your evaluation set is 100 examples, it cannot resolve anything smaller than about ten points, and you should say so out loud before running the comparison.

Lab 2. `pass@n` is `float(correct.any(0).mean())` — three lines given samples and labels. The diagnostic: if coverage ≈ accuracy, sampling more will help and your verifier is fine. If coverage accuracy, more samples are wasted money and every hour goes into the verifier. Our Panel B2 case was 0.8917 against 0.5011, which is as clear a signal as you will ever get.

Lab 3. `thinking_tokens` appears in `usage.output_tokens_details`; when streaming it arrives only on the final `message_delta`. Cost is `thinking_tokens × output_rate`, at the same rate as visible output. Section 13.8's worked example gives 9.9× on Opus 5 for an 8,000-token trace on a 500-token answer; substitute your own numbers and multiply by monthly volume.

Lab 4. Hold everything else fixed, including the prompt, and remember that changing effort invalidates prompt caching — so run the sweep as separate passes, not interleaved, or your cost numbers will be nonsense. Expect the interesting result to be that `medium` is indistinguishable from `high` on most non-agentic workloads, which is the documentation's own guidance for Sonnet 4.6 and a good hypothesis to test on anything else.

Lab 5. Ambiguous multi-class classification is the reliable source. Report the winning margin alongside accuracy: the failure mode you are demonstrating is 14-of-16 agreement on a wrong label. Once you have seen it on your own system you will not build an agreement-based confidence score again, which is the actual objective of the lab.

13.19 Best Practices

BEST PRACTICE

Buying test-time compute without wasting it

1. **Ask "what is the checker?" first.** If the answer is "a human" or "another model", expect a poor return and consider spending the money on a better model instead.
2. **Measure coverage before buying budget.** The gap between coverage and accuracy tells you whether to invest in sampling or in verification. They are different purchases.
3. **Pick the model, then the budget.** Panel E has no crossover. A better model raises coverage *and* selection; samples raise only coverage.
4. **Verify then vote.** Majority voting restricted to verifier-accepted candidates beat first-accepted by nearly nine points at zero extra token cost.
5. **Set `effort` explicitly.** The default is `high`. Silence is a decision to spend, on every request, forever.
6. **Hold effort and thinking configuration constant inside a cached conversation.** Both are rendered into the prompt; changing either invalidates the cache. Vary them across workloads, not within a session.
7. **Use `max_tokens` for the hard ceiling.** Effort is a behavioural signal and task budgets are advisory. Only `max_tokens` is enforced.
8. **Batch your parallel samples.** Sampling is embarrassingly parallel and rarely latency-critical; the Batch API is 50% off both directions.
9. **Log `thinking_tokens` from day one.** You cannot manage a line item you are not measuring, and it is not visible in the ordinary output count.
10. **Re-sweep effort when you change model.** The documentation says settings do not transfer. Treat carried-over parameters as untested.
11. **Set `display: "omitted"` when you do not show thinking to users** — it improves time-to-first-token when streaming. It does not reduce cost.
12. **Return thinking blocks unmodified inside a tool-use turn**, `redacted_thinking` included, and strip them when switching models mid-conversation.
13. **Prefer a tool to a trace for anything exact.** Chapter 10's ladder is unchanged: reasoning about arithmetic is worse and dearer than doing arithmetic.
14. **Publish the interval.** "Reasoning improved accuracy" without an interval and a sample size is not a finding; it is a hope with a number attached.

13.20 Common Mistakes

COMMON MISTAKE

Leaving effort at the default on a high-volume simple workload

Symptom. The bill rises by five to ten times after a model migration, with no measurable quality change. **Cause.** `effort` defaults to `high`, and on the 5-series thinking is on by default. Not setting the parameter is not the same as not using the feature. **Fix.** Sweep `low`, `medium`, `high` on your evaluation set, and pick the lowest level whose quality loss is not established. Disable thinking entirely where the model allows it and the task is simple. **Detection.** Log `usage.output_tokens_details.thinking_tokens` per request and alert on its ratio to visible output tokens.

COMMON MISTAKE

Sampling repeatedly with no diversity

Symptom. n samples, n times the cost, identical answers, zero gain. **Cause.** Temperature zero — or, on current Claude models, an assumption that sampling automatically varies when the sampling parameters are locked to their defaults and cannot be changed. **Fix.** Verify empirically that your candidates differ before building any selection stage on top of them. If they do not, parallel sampling is not available to you and knob 1 is your only option. **Detection.** Count distinct answers per problem. If the mean is near 1.0, stop paying for the extra samples.

COMMON MISTAKE

Reading agreement as confidence

Symptom. A confidence score built from self-consistency agreement is high on exactly the cases the system gets wrong. **Cause.** Agreement measures the sharpness of the model's distribution, not its correctness. A systematic error is a sharp, agreed-upon, wrong mode. **Fix.** Calibrate agreement against ground truth on a labelled set before using it as a signal, and expect it to fail on ambiguous and out-of-distribution slices. **Detection.** Plot accuracy against agreement. If the curve is flat or non-monotonic, the signal is not a confidence estimate.

COMMON MISTAKE

Buying more samples when the verifier is the bottleneck

Symptom. Accuracy flattens while cost keeps climbing linearly with n. **Cause.** Coverage is far above achieved accuracy. The right answer is already in the pool and selection is discarding it. Our own numbers: 0.8917 coverage, 0.5011 achieved. **Fix.** Measure coverage offline. If the gap is large, every engineering hour goes into the verifier and none into the budget. **Detection.** Track coverage and accuracy on the same labelled set at the same n. The gap is the diagnostic.

COMMON MISTAKE

Expecting reasoning to fix a structural limit

Symptom. Thinking budget raised repeatedly; accuracy on a specific task class stays at or near zero. **Cause.** Coverage is zero. Chapter 10's structural limits — exact arithmetic at scale, current external state, byte-level manipulation — are not degrees of difficulty that more tokens traverse. Panel D measured exactly 0.0000 at n=128. **Fix.** Chapter 10's compensating architecture: a tool, retrieval, or code execution. Not a budget. **Detection.** If `pass@n` is zero at n=32, no selection procedure will ever return a correct answer. Stop buying.

COMMON MISTAKE

Treating the thinking trace as an audit trail

Symptom. A compliance narrative, an explanation shown to a user, or an incident post-mortem built on the summarised thinking text. **Cause.** Conflating "text that causally influenced the answer" with "an account of why the answer was produced". Chapter 10's no-introspection limit and Chapter 5's "attention is not explanation" both apply, and the text you can read is a summary produced by a *different model* from the one that reasoned. **Fix.** Build audit trails from inputs, tool calls, tool results, retrieved documents and outputs — the things your system actually observed. Use the trace for debugging, never as evidence. **Detection.** Ask whether the artefact would survive someone showing that the trace omitted the deciding consideration. If not, it is not an audit trail.

COMMON MISTAKE

Changing effort or thinking configuration mid-conversation

Symptom. Cache hit rate collapses; input costs rise sharply on long sessions. **Cause.** The thinking configuration and resolved effort level are rendered into the prompt. Switching modes, changing `budget_tokens`, or changing effort all invalidate cache breakpoints — message-level always, tool and system breakpoints often. **Fix.** Choose a level at the start of a session and hold it. Vary across workloads, not within one. **Detection.** Watch `cache_creation_input_tokens` on requests that should have been reads. A non-zero value mid-conversation means something in the prefix changed.

COMMON MISTAKE

Setting a task budget too small

Symptom. The agent declines the task, scopes it down aggressively, or stops early with a partial result — behaviour that looks like a refusal or a bug. **Cause.** The documented failure mode. When Claude sees a budget clearly insufficient for the work, it may not start rather than start something it cannot finish. The minimum accepted value is 20,000 tokens, and the minimum accepted is not the minimum *sensible*. **Fix.** Measure your existing per-task spend without a budget, take the p99, and size from there. Raise the budget before debugging anything else. **Detection.** Correlate premature stops with the budget value. The countdown is invisible in the response, so the correlation is the only signal you get.

13.21 Performance Considerations

Thinking is a latency decision before it is a cost decision. Chapter 11's model — `latency ≈ prefill + output_tokens × per_token_decode`, decode at 82–99% of wall clock — says that adding 8,000 thinking tokens to a 500-token answer multiplies the dominant term by seventeen. Streaming does not help the way it usually does: the user sees nothing until thinking completes unless you surface the thinking text, and `display: "omitted"` exists precisely to shorten that dead time by skipping the streaming of thinking tokens. It improves time-to-first-*text*-token; it does not shorten the thinking.

Parallel sampling has the opposite profile. Wall-clock latency is one sample's latency, because the calls are concurrent; cost is `n` times the output plus a heavily discounted input, as section 13.8 priced at 5.3× for eight samples rather than eight. This makes best-of-n the *latency-cheap* and *money-expensive* form of test-time compute, and long thinking the reverse. Choose on which one binds.

Three operational limits from the documentation. Above roughly 32,000 thinking tokens per request, use the Batch API — long-running requests hit system timeouts and open-connection limits. The SDKs require streaming above `max_tokens` of 21,333. And thinking-heavy tasks routinely outlive the five-minute cache, which is the strongest argument for the one-hour duration: at a 2× write multiplier against 1.25×, converting a single miss into a hit pays for the difference immediately.

Finally, capacity. Chapter 9's KV-cache arithmetic is per-sequence and grows with sequence length; thinking tokens are sequence length. On keep-all models the accumulated thinking of previous turns stays resident, so a long agentic session's memory footprint grows faster than the visible transcript suggests. Chapter 9's concurrency ceilings move accordingly, and the fix is Chapter 54's compaction and thinking-block clearing rather than a bigger machine.

13.22 Security Considerations

The trace is generated text and inherits every property of generated text. If untrusted content enters the context, it can influence the reasoning exactly as it influences the answer. A reasoning model given a poisoned document may reason its way, at length and plausibly, into the attacker's conclusion — and the length and plausibility make the output *more* convincing, not less. Chapter 58 covers prompt injection properly; the relevant point here is that a longer trace is a larger attack surface for a persuasion attack, not a defence against one.

Do not show raw reasoning to users by default. Anthropic returns summarised rather than raw thinking, and the documentation says plainly that this is to provide the intelligence benefit while preventing misuse. Your own reasoning traces deserve the same treatment: they may contain intermediate speculation, rejected approaches, restatements of confidential context, and material that reads badly out of context. `display: "omitted"` is a reasonable default for a user-facing product.

Thinking blocks are signed, and the signature is not yours to interpret. The `signature` field is an encrypted copy of the full reasoning that the API uses to verify a block was generated by Claude. It is opaque: do not parse it, do not attempt to derive anything from it, and pass it back exactly as received. It is also compatible across the Claude API, Amazon Bedrock and Google Cloud, which matters for multi-platform deployments. Modified blocks are rejected with a 400, and that rejection is a feature — it is what prevents a forged reasoning trace from being injected into a conversation.

Cached thinking is retained context. Thinking blocks cached alongside tool results count as input tokens when read back, and on keep-all models they persist across turns. Anything sensitive that entered the reasoning is therefore resident for the life of the cache. Chapter 61's data-residency and retention obligations apply to thinking content exactly as to any other context, and Chapter 11's `inference_geo` multiplier applies to the tokens.

Reasoning is not an explanation and must not be used as a control. This is the governance point and it is worth stating flatly. If a decision needs to be justifiable — a credit refusal, a clinical suggestion, a moderation action — the justification cannot be the model's trace, because the trace has no guaranteed relationship to the computation and is, in the summarised case, produced by a different model. Justify decisions with the inputs, the retrieved evidence, the tool results and the deterministic rules your system applied. Fable 5's `reasoning_extraction` refusal category is the platform declining to pretend otherwise, and it is the right default.

13.23 Real Production Examples

Coding agents. The canonical good case, and the reason reasoning models and coding agents arrived together. The verifier is a compiler and a test suite: fast, sound within its coverage, and already written. Coverage is high because a competent model can produce a working patch given attempts. Every ingredient in section 13.6's table is at the favourable end, and the introduction's 38%-to-71% is the shape of result to expect.

Structured extraction with a schema. Chapter 10's constrained decoding guarantees the *shape* of the output for free — no sampling required — so the interesting verifier is semantic: cross-field consistency, totals that must add up, dates that must be ordered. These checks are cheap and deterministic, which makes modest best-of-n worthwhile. Note the documented incompatibility: structured outputs are incompatible with citations, and with message prefilling, which thinking also forbids.

Retrieval-grounded question answering. The hard middle case. Chapter 8 established why lexical grounding checks pass the most dangerous fabrications: a fabricated claim assembled from words that do appear in the source will satisfy a token-overlap check. So the natural cheap verifier here is *unsound in exactly the direction that matters*, and best-of-n over it will confidently select fluent fabrications. Citations help — the pointers are guaranteed valid and `cited_text` costs no output tokens — but a valid pointer is not a valid inference. Treat this as an uncheckable workload until you have measured your checker against labelled fabrications.

High-volume classification and routing. Reasoning is a tax. Low effort, thinking off where permitted, `n = 1`. The documentation's own recommendation for subagents and simple classification is `low`. If accuracy is inadequate, the answer is a better model or better labels, not a longer trace — and Haiku 4.5 at $1/$5 with `n = 1` will beat Opus 5 at high effort on cost by two orders of magnitude while quite possibly matching it on the task.

Long-horizon agentic work. Where `xhigh` effort and task budgets earn their place: multi-hour runs, millions of tokens, many tool calls. The pattern that works is effort for depth, task budget for breadth, `max_tokens` for the hard ceiling, and interleaved thinking so the agent reasons about each tool result before choosing the next call. Part 9 builds these systems properly; Chapter 50 covers the orchestrator–subagent topology in which `low` effort on subagents and `high` on the orchestrator is a standard and effective split.

13.24 Certification Notes

Concept	Exam domain	What is likely to be asked
Adaptive vs extended thinking	D4 Prompt Engineering & Structured Output	Which models take which; that `type: "enabled"` returns 400 on 4.7+
The `effort` parameter	D4, D1	Five levels; `high` is the default; it governs all tokens, not just thinking
Effort is not a cap	D5 Context Management & Reliability	`max_tokens` is the only enforced limit

Concept	Exam domain	What is likely to be asked
Thinking billed as output	D5, D1	Output is 5× input on every current model; thinking is not cacheable
Thinking and the context window	D5	Current-turn thinking counts toward `max_tokens`; prior-turn preservation is per-model
Thinking block signatures	D2 Tool Design & MCP Integration	Must be returned unmodified in a tool-use turn; `redacted_thinking` too
Interleaved thinking	D2, D1	Automatic with adaptive; Haiku 4.5 does not support it; beta header only in manual mode
Task budgets	D1 Agentic Architecture & Orchestration	Beta header, 20,000-token minimum, advisory not enforced, not on Sonnet 5
Verifier asymmetry	D1, D5	Where test-time compute pays; coverage versus selection
Test-time compute vs structural limits	D1	Amplification, not creation; the fix for a structural limit is a tool
Caching interaction	D5	Any thinking or effort change invalidates cache breakpoints
Sampling parameter restrictions	D4	Non-default `temperature`/`top_p`/`top_k` return 400 on current models

CERTIFICATION NOTE

Exam Tips

- **"Which is the default effort level?"** `high`. Setting it explicitly to `high` is identical to omitting it. This is a favourite.
- **Adaptive is a thinking *mode*; the effort scale is** `low/medium/high/xhigh/max`**.** Passing `adaptive` as an effort value is a documented error, which makes it a good distractor.
- `type: "enabled"` **with** `budget_tokens` **is legacy.** Deprecated on 4.6, rejected with a 400 on 4.7 and later, and the only mode available on Haiku 4.5 and other 4.5-generation models.
- **Task budgets: advisory, 20,000 minimum, beta header** `task-budgets-2026-03-13`**, not supported on Sonnet 5.** All four are testable facts.
- **Thinking tokens are billed as output tokens** even when `display` is `omitted`. Omitting buys latency, never cost.
- **Coverage bounds accuracy.** If a question describes flat accuracy with rising cost, the answer is about the verifier, not the budget.
- **If a question describes zero accuracy that does not respond to any budget**, it is describing a structural limit and the answer is a tool.
- **Watch for "thinking is a record of the model's reasoning".** It is not. Any option offering the trace as an audit trail or explanation is wrong.

13.25 Twenty Practice Questions

1. What is test-time compute, and how does it differ from training compute?
2. Which three knobs make up test-time compute, and which one generates no tokens?
3. Define coverage and selection, and state how they combine into accuracy.
4. A workload shows coverage of 0.90 and accuracy of 0.52 at n=32. What should you buy?
5. Why does test-time compute fail entirely on a structural limit?
6. What is the default `effort` level on the Claude API?
7. Which models reject `thinking: {"type": "enabled"}` with a 400 error?
8. Are thinking tokens billed as input or output?
9. Does `display: "omitted"` reduce cost, latency, or both?
10. What does the `signature` field on a thinking block contain, and what must you do with it?
11. Which current models do **not** support task budgets?
12. What is the minimum accepted `task_budget.total`, and what happens below it?
13. Is a task budget enforced? What is enforced?
14. Why does changing `effort` mid-conversation raise your input costs?
15. On which models do prior turns' thinking blocks remain in context by default?
16. Why does self-consistency fail when a model's errors are systematic?
17. What makes casting out nines a verifier rather than a solver?
18. Why does an imperfect verifier's loss *grow* as the sample budget grows?
19. Which sampling parameters return a 400 error on Claude Opus 5, and under what conditions?
20. When is a second model as judge a poor verifier choice, and what should you check first?

13.26 Answers and Explanations

1. **Test-time compute** is computation spent producing an answer to a single request — tokens generated, samples drawn, seconds spent — as opposed to training compute, which is spent once before serving. The distinction that matters commercially is that you choose test-time compute per request and pay for it forever. *Wrong answers usually confuse it with "the model thinking harder because it is bigger", which is training compute.*
2. Think longer (sequential tokens), sample more (parallel attempts), and verify-and-select. **Verify-and-select generates no tokens** and is where most of the value sits. *A distractor here is "prompt engineering", which is not a compute knob at all.*
3. **Coverage** is the probability at least one of n candidates is correct — a property of model and sampling. **Selection** is the probability your procedure returns a correct candidate given one exists — a property of your verifier. `accuracy ≈ coverage × selection`. *"Coverage means how much of the input the model read" is the common wrong reading.*
4. **The verifier.** Coverage of 0.90 means the right answer is in the pool nine times in ten; accuracy of 0.52 means you are discarding it. More samples raise the number you are already failing to use. *"More samples" and "a longer thinking budget" both raise coverage, which is not the binding constraint.*
5. Because a structural limit means coverage is **zero**: the model cannot produce the correct answer at any temperature in any number of attempts, so selection multiplies by zero. Panel D measured

exactly 0.0000 at n=128. *"Because the model runs out of context" is wrong — it would fail identically with unlimited context.*

6. `high`**.** Setting it explicitly to `high` produces exactly the same behaviour as omitting the parameter. `medium` *is the plausible distractor; it is the documented recommendation for Sonnet 4.6, not the API default.*
7. **Claude 4.7 and later**, including Opus 4.7, Opus 4.8, Opus 5, Sonnet 5, Fable 5 and Mythos 5. It is deprecated but still functional on the 4.6 generation, and it is the only mode on 4.5-generation models. *"All models" is wrong; Haiku 4.5 requires it.*
8. **Output**, at the same rate as visible output text — five times the input rate on every current model — whether or not the thinking text is returned. *"Input, because it is internal" is the trap.*
9. **Latency only.** The documentation is explicit that you are still charged for the full thinking tokens; omitting skips streaming them, which improves time-to-first-text-token. *"Both" is the intuitive and wrong answer.*
10. An **encrypted copy of the full reasoning**, used by the API to verify the block was generated by Claude. Pass it back **unchanged**; it is opaque and must not be parsed. Modified blocks are rejected with a 400. *"A hash you can use to deduplicate" is wrong.*
11. **Claude Sonnet 5**, Opus 4.6, Sonnet 4.6 and Haiku 4.5. Supported in beta on Opus 5, Fable 5, Mythos 5, Opus 4.8 and Opus 4.7. Sonnet 5 is the one people get wrong, because it supports `xhigh` effort and the two features feel related. *They are not: effort tunes depth, task budgets tune breadth.*
12. **20,000 tokens** on every model that currently supports the feature. Below it, a **400 error**. *Note also that the minimum accepted is not the minimum sensible; an undersized budget causes refusal-like behaviour.*
13. **No — task budgets are advisory.** Claude may exceed one rather than interrupt an action mid-flight. `max_tokens` **is the enforced limit**, truncating with `stop_reason: "max_tokens"`. *Effort is likewise a behavioural signal, not a cap.*
14. Because the resolved effort level is **rendered into the prompt**, so changing it starts a new cache prefix. Message-level breakpoints always miss; tool and system-prompt breakpoints may. *"Because higher effort means more input tokens" is wrong — the input is unchanged; the cache is what breaks.*
15. **Opus 4.5 and later Opus models, Sonnet 4.6 and later Sonnet models, Fable 5 and Mythos 5** keep all prior turns and bill them as input. Earlier Opus and Sonnet models, and every Haiku through 4.5, keep only the last turn. *This is the fact that decides whether long agentic sessions accumulate context.*
16. Because voting returns the **mode** of the sampled distribution, and a systematic error *is* the mode. More samples estimate the mode more precisely, so the procedure converges to the wrong answer faster. Agreement measures sharpness, not correctness. *"Because the samples are correlated" is a partial answer; the sharper statement is that the mode itself is wrong.*
17. Because it is **strictly cheaper than solving the problem** — one modulo per operand and a digit sum — while being **independent** of the mechanism that produced the answer. A "verifier" that recomputes the answer from scratch has moved the work rather than checked it. *It is also incomplete, which is realistic and measurable: our false-accept rate was 0.0742.*
18. Because each additional candidate is another opportunity for a **wrong-but-accepted** answer to be encountered before a correct one. Coverage rises, but so does the number of ways to select badly. Our

measured loss grew monotonically from 0.0042 at n=2 to 0.3906 at n=128. *This is why budget cannot fix a selection problem.*

19. `temperature`, `top_p` **and** `top_k` — non-default values return 400 on Fable 5, Mythos 5, Opus 5, Opus 4.8, Opus 4.7 and Sonnet 5, **on every request, whether or not thinking is used**. On older models the restriction applied only while thinking was active. *The "only while thinking is on" answer is right for the previous generation and wrong for the current one.*
20. When a cheap deterministic check exists, or when the judge is no more reliable than the generator. A judge **doubles the bill** and caps selection at its own accuracy. Check first with Chapter 8's Cohen's kappa against human labels; if agreement is poor, the judge is adding cost and noise. *"Never" is too strong — a judge is the right answer for genuinely subjective criteria, once measured.*

13.27 Ten Practical Exercises

1. Reproduce Panel B on your own hardware and confirm the best-of-64 and best-of-128 intervals overlap.
2. Add a `pass@k` column for `k < n` — accuracy if you could inspect only the first `k` of `n` samples — and find the knee.
3. Replace casting out nines with a mod-11 check and measure whether the higher modulus buys selection accuracy.
4. Combine mod-9 and mod-11 and measure the compound false-accept rate against the product of the individual rates.
5. Implement weighted majority voting using the model's own sequence probability as the weight, and compare against unweighted.
6. Measure the correlation between samples: what fraction of problems produce fewer than three distinct answers at n=16?
7. Add a per-problem adaptive budget — stop sampling as soon as two candidates agree *and* pass the verifier — and report the token saving.
8. Train a third model at h=64 and locate the exact parameter count at which best-of-8 overtakes the next size up at n=1.
9. Rerun Panel C with a deliberately biased sampler and demonstrate majority voting converging to the wrong answer.
10. Price every row of Panel B against the real Opus 5 rate card, with and without prompt caching, and identify the cheapest row above 0.45.

13.28 Five Challenge Labs

1. **Build a process verifier.** Instead of checking the final answer, check each intermediate step. Measure whether step-level verification beats answer-level verification at equal cost, and report where it does not.
2. **Learn a verifier.** Train a small classifier to predict correctness from a candidate answer alone, without the operands. Measure its precision, then measure how much selection accuracy it actually buys — and be prepared for the answer to be "less than the modulo check".

3. **Adaptive budget allocation.** Given a fixed total token budget across a batch of problems, allocate samples per problem to maximise total accuracy. Compare against uniform allocation. The optimal policy needs a difficulty estimate; measure what a bad one costs you.
4. **Reproduce the crossover, honestly.** Find a pair of model sizes and a task where spending test-time compute on the smaller model *does* beat the larger one at equal FLOPs. Report the conditions under which it happens, and the conditions under which your Panel E result recurs.
5. **The uncheckable benchmark.** Take a genuinely subjective task, build a model-as-judge verifier, measure the judge's agreement with human labels using Cohen's kappa, and then compute how much selection accuracy that kappa permits. Show the arithmetic connecting judge reliability to end-to-end accuracy.

13.29 Ten Interview Questions

1. What is test-time compute, and why is it described as a second scaling axis?
2. Mechanically, why does generating intermediate tokens improve a fixed-depth network's answers?
3. How does a reasoning model differ from prompting a general model to think step by step?
4. Explain coverage and selection, and how you would measure each.
5. A team reports that thinking tripled their bill and did nothing for accuracy. What do you ask them first?
6. When does self-consistency fail, and what would you measure to predict it?
7. Why is a second model as judge usually a poor verifier?
8. Thinking tokens are billed as output. Walk me through the cost of an 8,000-token trace on a short answer.
9. Under what circumstances would you choose a bigger model over more samples?
10. Why should a reasoning trace not be used as an audit trail?

13.30 Summary

Test-time compute is the second scaling axis, and it is the one you own. Chapter 12's laws relate training loss to training resources; every term in them is fixed by the time a request arrives. Reasoning models moved the remaining variable to inference: for the same frozen weights, spending more tokens on a single request measurably improves the answer. That is a genuine change in what more compute buys, and Chapter 12 could not have predicted it, because a scaling law tells you what happens along the axes you varied and nobody varied this one. The mechanism is unremarkable once stated. Intermediate tokens are generated, appended, and conditioned on, through the same decode loop, the same attention and the same KV cache as everything else. They add serial computational depth to a network whose layer count is fixed, they externalise state into the only memory the model has, and they shift the conditional distribution towards the answers that usually follow careful working. What they do not do is explain anything: Chapter 10's no-introspection limit is unrepealed, and a trace is text that influenced the answer rather than an account of how the answer was computed.

There are exactly three knobs — think longer, sample more, verify and select — and the third one generates no tokens and holds nearly all of the value. That is because accuracy factors into coverage and

selection. Sampling and thinking raise coverage, the chance that a correct answer exists somewhere in what you generated. Only verification raises selection, the chance that you return it. This factorisation is the chapter's most portable idea: it converts "should we turn on reasoning" from an opinion into two measurements, and it tells you which one to spend on. Where the gap between coverage and accuracy is large, buying budget is buying options you cannot exercise.

Our harness measured all of it on one task, with every strategy seeing identical samples so the comparison was exact. Best-of-8 tripled a weak model's accuracy, from 0.1308 greedy to 0.3535, which is test-time compute working. Then the curves parted: best-of-n flattened after n=32 while coverage kept climbing to 0.8917, and the doubling from 64 to 128 samples produced a gain of 0.0031 that its own confidence interval could not distinguish from nothing. Handing the harness a perfect verifier resolved the ambiguity — best-of-n became coverage exactly, and the 0.3906 difference at n=128 was a clean measurement of correct answers the checker was throwing away. That loss grew monotonically with budget, which is the formal reason more samples cannot repair a weak verifier. Removing the verifier entirely priced the asymmetry at 37.3 points on identical tokens, and produced the result we did not want: self-consistency at sixty-four samples scored 0.1245 against a single greedy sample's 0.1308. Majority voting lost to doing nothing, at sixty-four times the cost, at every budget we measured. We publish it because voting returns the mode of the sampled distribution and greedy decoding estimates the same mode more sharply for free — and because agreement, which people reach for as a confidence signal, measures the sharpness of a distribution rather than its correctness.

Two boundaries close the experimental argument. On five-digit operands, where Chapter 10 established the network learned four columns rather than an operation, coverage over 128 samples was exactly zero and every strategy returned nothing. Test-time compute amplifies a capability; it does not manufacture one, and the fix for a structural limit remains a tool. And on the crossover: at equal compute the small model scored 0.1495 where the large model scored 0.9047, and at fifty-three times the compute it reached 0.5011 and never came close. On our task there was no budget at which samples substituted for parameters — a narrower claim than the literature's general position, and one we state with its conditions, because our two models straddle the steepest part of Chapter 12's curve. The constructive half of the same table is that the *large* model's best-of-8 beat its own greedy decoding by five and a half points. The ordering rule follows from both halves: pick the model, then buy samples, then improve the verifier, checking after each step.

The economics make all of this urgent rather than academic. Thinking tokens are billed as output, output is five times input on every current Claude model, and thinking is not cacheable because there is no prefix to hit. An 8,000-token trace on a 500-token answer is a 9.9× bill on Opus 5, and at a million requests a month that is the difference between $22,500 and $222,500 — which is the introduction's summariser, explained. Effort defaults to `high`, thinking is on by default on the 5-series, and changing either mid-conversation invalidates the prompt cache you were relying on. Parallel sampling prices better than people expect, at 5.3× for eight samples once caching amortises the input, and the Batch API halves it again. The knobs are `effort` across five levels, task budgets in beta for the breadth of an agentic loop, `max_tokens` as the only hard ceiling, and a set of compatibility rules — signed thinking blocks returned unmodified, per-model preservation defaults, sampling parameters that now return 400 on every request — that are as much a part of the design as the accuracy numbers. Part 2 ends here. What a model is, what it cannot do, what it costs, how it scales and how it spends compute at inference: from Chapter 14 onwards these stop being objects of study and become a component you select, configure and

hold to a budget.

KEY TAKEAWAYS

1. **Test-time compute is the second scaling axis** — spent per request, by you, forever. Chapter 12's laws do not cover it.
2. **A reasoning trace is generated tokens conditioned on**, adding serial depth and external state. It is not an explanation.
3. **Three knobs: think longer, sample more, verify and select.** The third generates no tokens and holds the value.
4. `accuracy ≈ coverage × selection`. Measure both; the gap tells you what to buy.
5. **The verifier asymmetry is worth 37.3 points** on our measurements — identical samples, identical tokens, checker or no checker.
6. **An imperfect verifier gets worse with budget**: 0.0042 lost at n=2, 0.3906 at n=128.
7. **Self-consistency lost to greedy** — 0.1245 against 0.1308 at 64× the cost. Agreement is not correctness.
8. **Coverage zero means budget zero.** 0.0000 at n=128 on a structural limit; the fix is a tool.
9. **No crossover on our task**: 0.5011 at 53× the compute against 0.9047 at 1×. Pick the model first.
10. **Thinking is billed as output at 5× input and is not cacheable.** 9.9× on a short answer. `effort` defaults to `high`.

13.31 Glossary

Term	Definition
Adaptive thinking	`thinking: {"type": "adaptive"}`; Claude decides whether and how deeply to think, steered by `effort`.
Best-of-n	Sample n candidates, return one a verifier accepts.
Casting out nines	An incomplete arithmetic check: a correct sum matches the operands' sum modulo nine.
Chain of thought	Intermediate generated tokens that the final answer is conditioned on.
Coverage (`pass@n`)	The probability at least one of n candidates is correct. An oracle quantity.
Effort	`output_config.effort`; five levels from `low` to `max`, default `high`, governing all tokens.
Extended thinking	Legacy manual mode: `type: "enabled"` with `budget_tokens`. Rejected on Claude 4.7 and later.
Interleaved thinking	Thinking between tool calls within one assistant turn; automatic under adaptive thinking.
Reasoning model	A model trained by RL on final-answer correctness with intermediate tokens left free.
Redacted thinking	A `redacted_thinking` block returned when reasoning is safety-redacted; round-trip unchanged.
Selection	The probability of returning a correct candidate given one is present. A property of the verifier.

Term	Definition
Self-consistency	Return the modal answer across n samples. Needs no verifier, and usually shows it.
Signature	The encrypted full reasoning attached to every thinking block; opaque, and returned unmodified.
Summarized thinking	The readable thinking text, produced by a different model; billed at the full thinking token count.
Task budget	An advisory token allowance for a whole agentic loop; beta, 20,000-token minimum.
Test-time compute	Computation spent producing an answer to a single request.
Verifier	A procedure cheaper than solving the problem that decides whether a candidate answer is acceptable.

13.32 Further Reading

Primary sources. The platform documentation is the authority for everything in sections 13.10 to 13.13, and it changes: `platform.claude.com/docs/en/build-with-claude/thinking` for the mode-neutral mechanics, `.../extended-thinking` for the legacy manual mode and the migration mapping, `.../effort` for the five levels and the per-model recommendations, and `.../task-budgets` for the beta. Re-verify before quoting any of it. On the research side, Wei et al., *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models* (2022) is where the prompted form was named; Wang et al., *Self-Consistency Improves Chain of Thought Reasoning* (2022) is the technique this chapter reports a negative result against, and reading it alongside our Panel C is instructive about how much task and temperature matter. Cobbe et al., *Training Verifiers to Solve Math Word Problems* (2021) is the origin of learned verifiers and of the best-of-n framing. Snell et al., *Scaling LLM Test-Time Compute Optimally* (2024) is the closest thing to a scaling law for this axis and is where the crossover question is posed properly. Lightman et al., *Let's Verify Step by Step* (2023) is the process-supervision result behind Challenge Lab 1. For the honesty question, Turpin et al., *Language Models Don't Always Say What They Think* (2023) demonstrates traces that omit the deciding consideration, and should be read by anyone tempted to use a trace as an audit trail.

Where to go next in this book. Part 3 begins at Chapter 14 with the Claude model family, and Chapter 15 turns this chapter's cost arithmetic and Chapter 12's capability arithmetic into a selection procedure. **Chapter 18 is the direct sequel**: extended and adaptive thinking as an API surface, in full, including the streaming events and block handling this chapter deliberately left alone. Chapter 19 covers rate limits and quotas, which is where a thinking-heavy workload meets its ceiling. Chapter 24 builds the regression suites that every measurement here belongs in, and Chapter 30 covers batch processing and prompt caching properly — the two features that make parallel sampling affordable. Chapter 34's error contracts and Chapter 35's tool chaining are where interleaved thinking becomes an architecture rather than a parameter. Chapter 51 covers reflection and self-healing, which is test-time compute organised as an agent loop rather than a request. Chapter 54 covers compaction and thinking-block clearing, the answer to accumulated thinking on keep-all models. Chapter 57 is the observability that makes the ledger in section 13.16 a production artefact rather than an exercise.